Object Oriented Programming

Java

Non Access Modifiers

- final
- static
- abstract
- strictfp
- native
- synchronized
- transient
- volatile

Static

- **Static keyword can be used for:**
  - Variable
  - Method
  - Class
- ***Why do we need static methods?***

It becomes necessary to call methods **without the need for an object** of the class to exist or sometimes a method performs a task that does not depend on the contents of any object.

It is called static method or class method.

Static

Example: we can calculate the square root of 900.0 with the static method call

```
Math.sqrt( 900.0 );
```

Why Is Method main Declared static?

When you execute the Java Virtual Machine (JVM) with the java command, the JVM attempts to invoke the main method of the class you specify—when no objects of the class have been created. Declaring main as static allows the JVM to invoke main without creating an instance of the class.

Static Class

- Classes can also be made static in Java.
- Java allows us to define a class within another class. Such a class is called a nested class.
- The class which enclosed nested class is known as Outer class. In java, we can't make Top level class static. ***Only nested classes can be static***.

Static Class Example

```
class OuterClass{
    private static String msg = “Hello World";
    // Static nested class
    public static class NestedStaticClass{
     // Only static members of Outer class is directly accessible in nested  static class
            public void printMessage() {
            // Try making 'msg' a non-static variable, there will be compiler error
                    System.out.println("Message from nested static class: " + msg);
            }
    }
}
```

Static - Rules

- A method marked as static can only access other static methods and variables directly.
- To access the instance variables and methods, a static method should have an instance on which the object members should be invoked.
- Static methods cannot be overridden.
- Static methods can be overloaded.
- Methods that differ only by **static** keyword cannot be declared.

Static (Example)

```
public class StaticClass {
    private String name = "CSE360";
    private static String msg = “Java Technology";

    public static void main(String[] args) {
        System.out.println(name);
        System.out.println(msg);
    }
}
```

Static

- ***Why static methods cannot access non-static variables or methods?***
  - Static variables **do not belong** to any particular instance of the class.
  - They are recognized with the name of the class.
  - Static methods are loaded at **Compile time** and non-static things at **Run time**.
  - While compiling, if you call a non-static data member, it would need an object which is created at Run time.

  Suppose you call Person.getExpense() and getExpense is a static method.

  If you use non-static variables inside the method, how would it know which variables to use?

Final

- **Final keyword can be used for:**
  - Variable
  - Method
  - Class

Final Variable

- A final variable can be explicitly initialized **only once**.
- A reference variable declared as final can never be **reassigned** to refer to an different object.
- The data within the object can be changed. So the state of the object can be changed but not the reference.

Final Method

- A final method **cannot be overridden** by any subclasses.
- The final modifier prevents a method from being modified in a subclass.

Final Class

- The main purpose of using a class being declared as final is to prevent the class from being derived.
- If a class is marked as final then **no class can inherit** any feature from the final class.

Final (Example)

```
public static void main(String[] args) {
        Outerclass obj1 = new Outerclass();
        Outerclass obj2 = new Outerclass();
        final Outerclass obj;
        obj = obj1;
        obj = obj2;          Compiler error!!
 }
```

Abstract

- Abstract can be applied for classes and methods only.
- When a method is marked as abstract – It should not have implementation. E.g., public abstract void move();
- Abstract methods should end with ‘;’ and not with ‘{ }’.

Abstract - Rules

- When a method marked as abstract, the whole class should be marked as abstract.
- A class can be abstract without any abstract methods in it.
- We cannot create an instance of abstract class. Why?
  - An abstract class is not complete! Someone marked it abstract to tell you that some **implementation is missing in the code**.
  - The **abstract** keyword ensures that no one would accidentally initiate this incomplete class.
- An abstract method should be overridden in the subclass or should be marked as abstract.

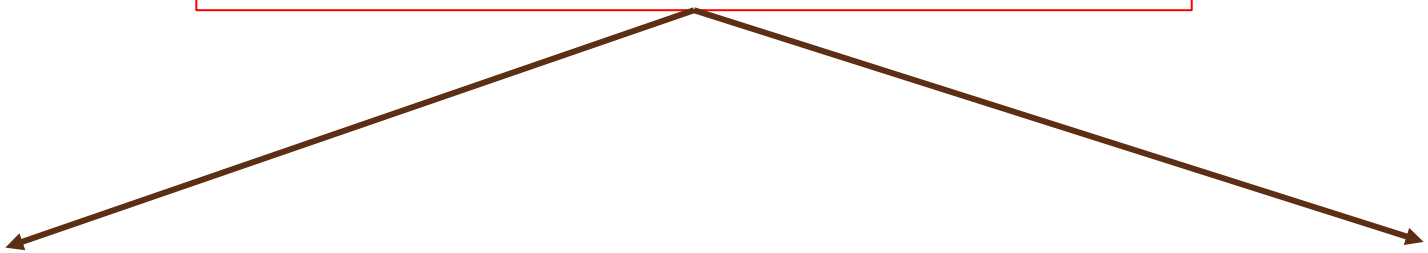
Abstract - Rules

- Abstract classes can have concrete(non- abstract) methods in it.
- Abstract methods cannot be marked as final.
- Abstract classes cannot be marked as final.
- Abstract methods cannot be static.
  - If a method is static, it must be accessible via just the class name. But we can't invoke a method that has no implementation.
- Abstract methods cannot be private.

Abstract (Example)

Person.java

```
public abstract class Person {
    private String name;
    public double expense;
    public abstract double getExpense(int month);
}
```



Teacher.java

```
public class Teacher extends Person{
    public double getExpense(int month){
        expense = 3500*month;
        return expense;
    }
}
```

Student.java

```
public class Student extends Person{
    public double getExpense(int month){
        expense = 250*month;
        return expense;
    }
}
```

Synchronized

- Synchronized can only be applied for methods.
- Any method or block that is synchronized will only allow one single thread to execute the code at a given time.
- Synchronized can be used with any access modifier.
- Example:

```
public synchronized add(){
        ……
}
```